COS 350 Lecture 13

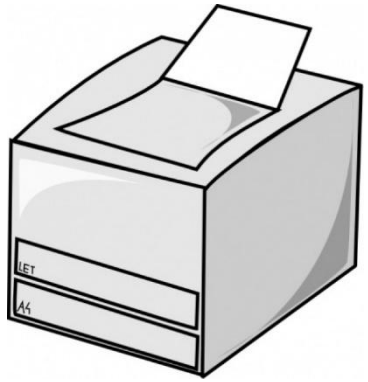
devices

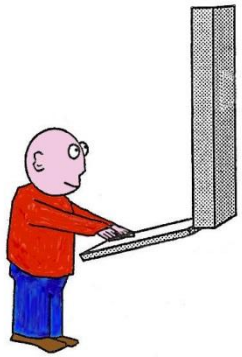
unix: write

special device properties:

- terminals: tcgetattr(), tcsetattr()
- files: fcntl()
- others: ioctl()

Unix treats all devices like files:
open(), close(), read(), write(), …





```
ls /dev
who
tty
ls -l /dev/pts
write, mesg
```

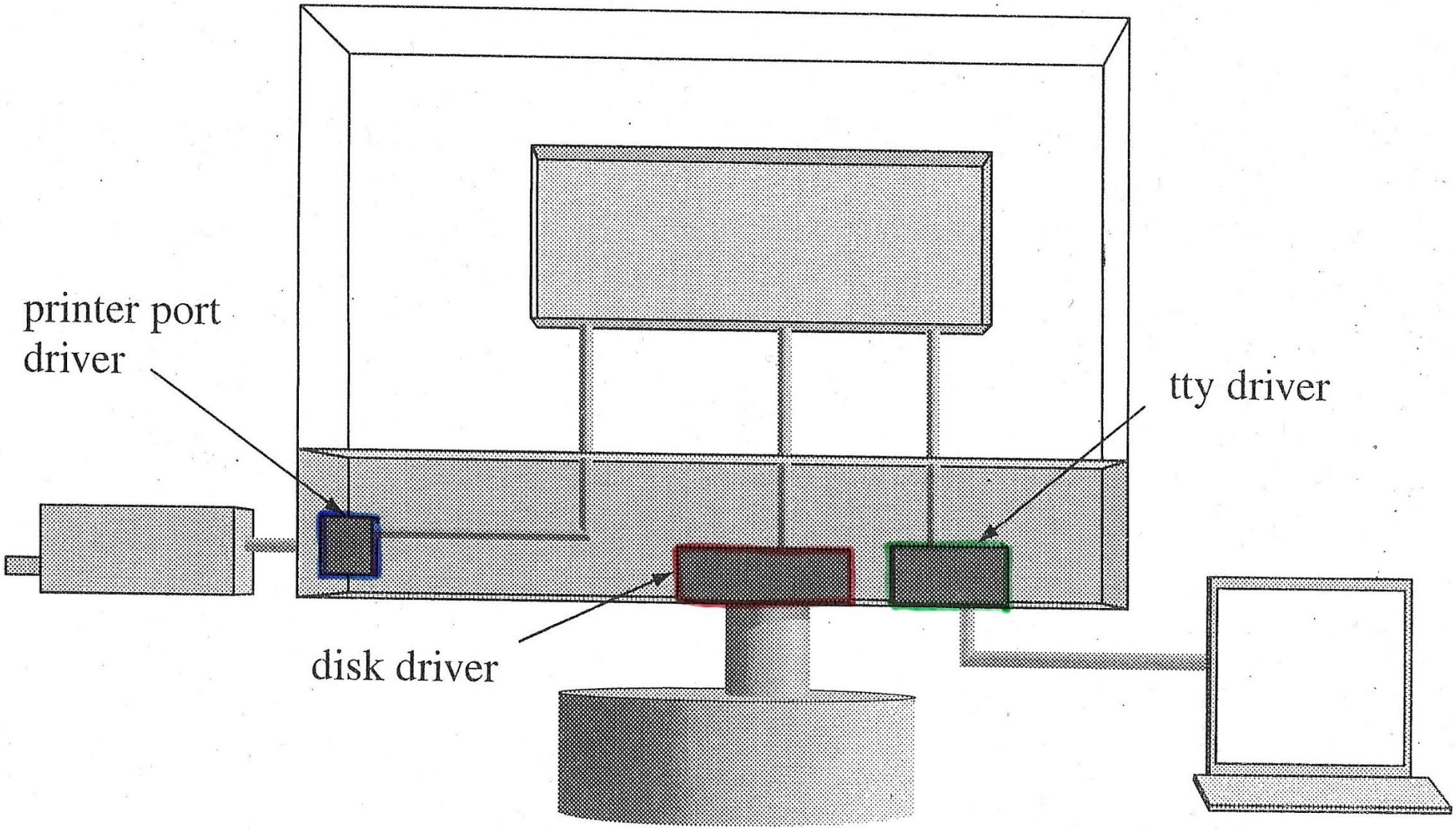
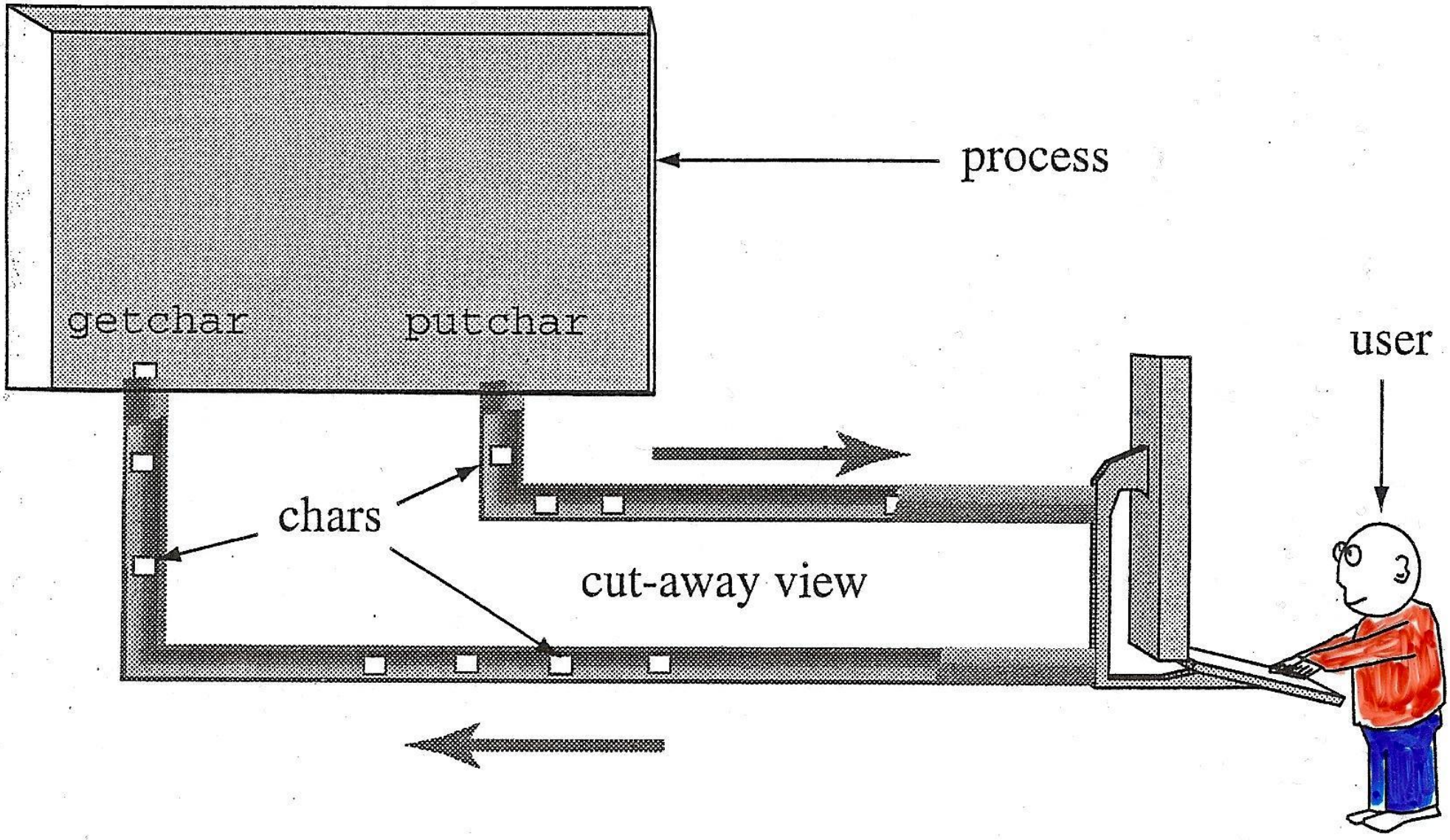
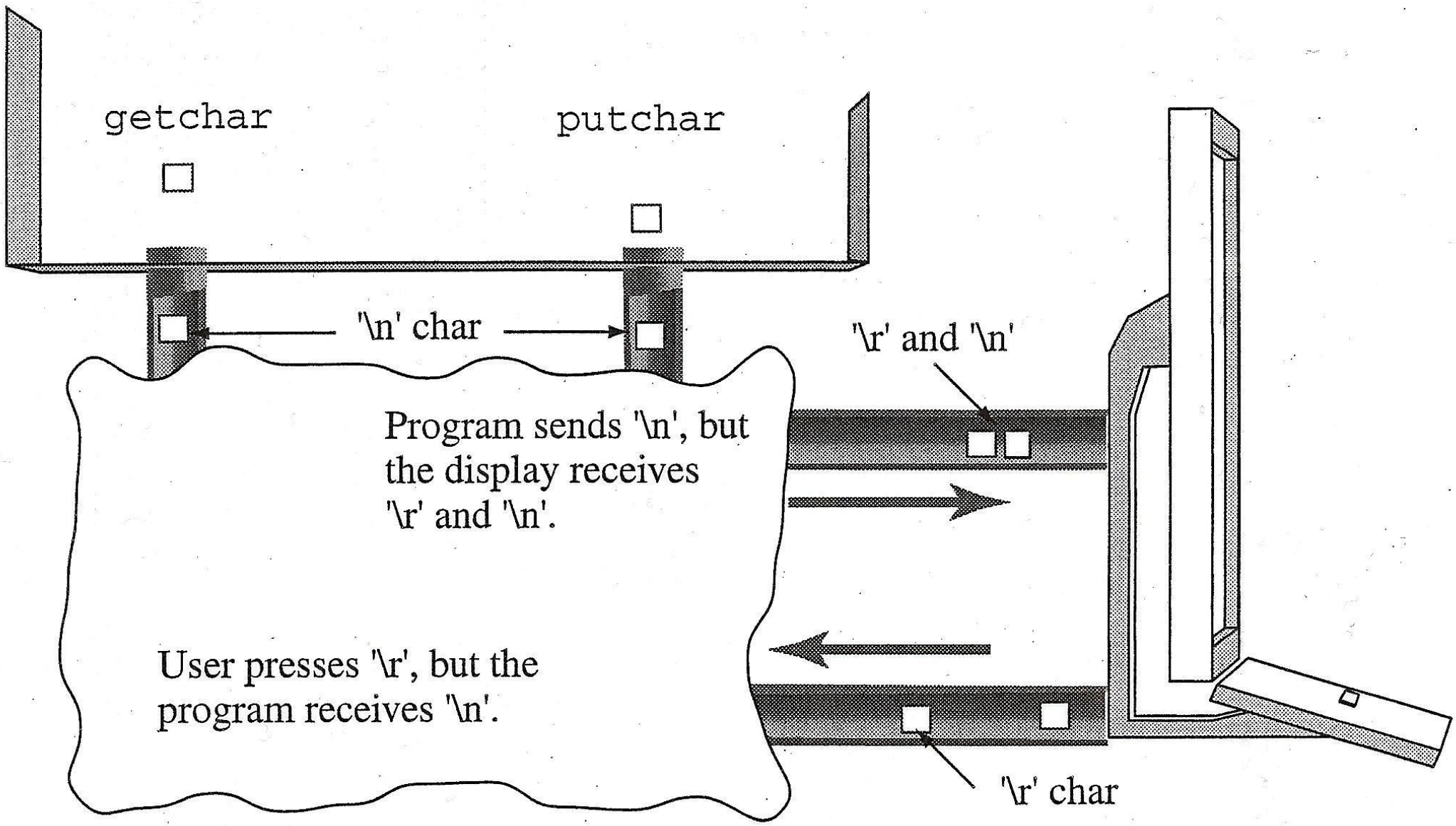
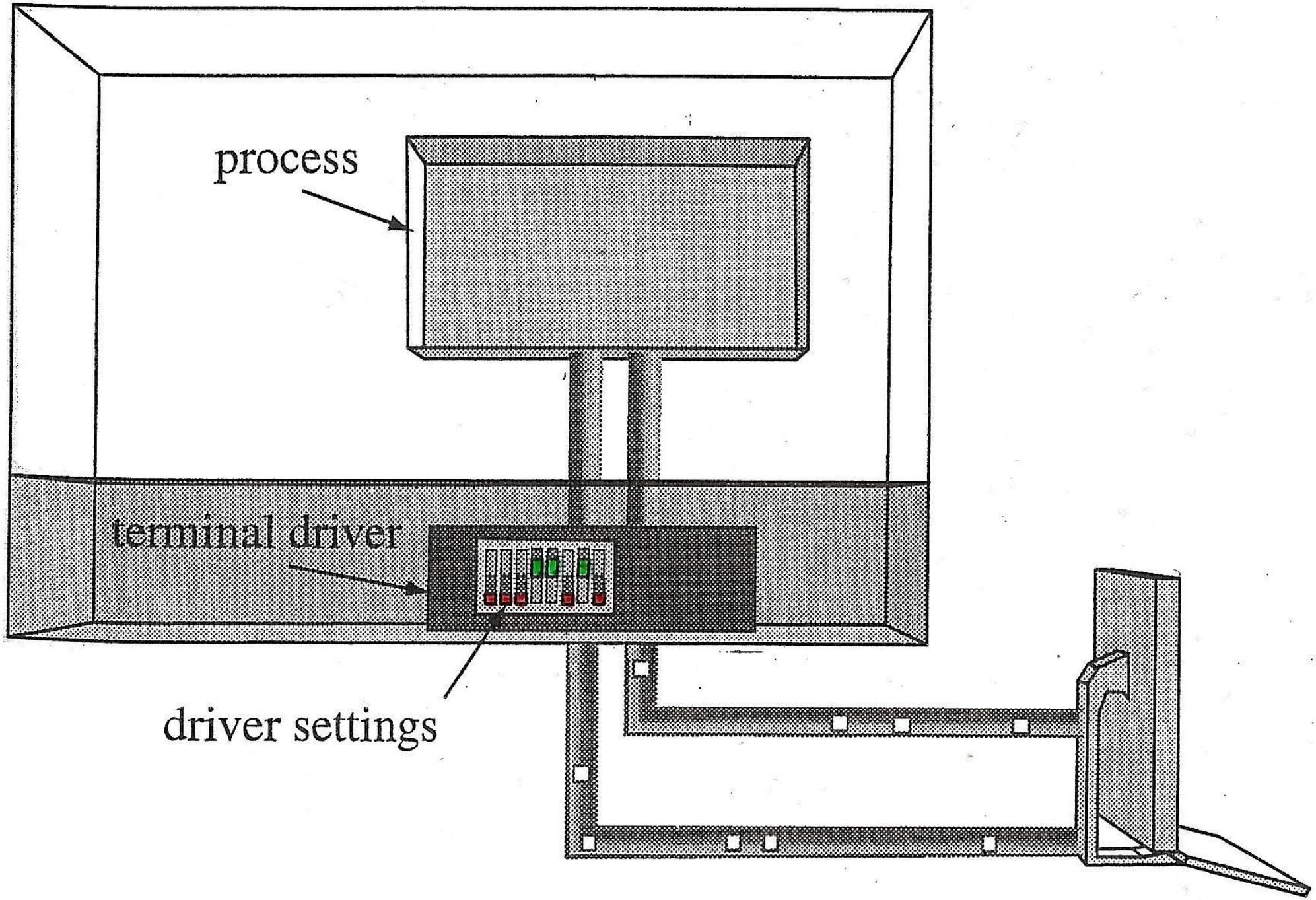


FIGURE 5.13

File descriptors, connections, and drivers.


process
getchar
putchar
user
chars
cut-away view


getchar
putchar
'\n' char
'\r' and '\n'
Program sends '\n', but the display receives '\r' and '\n'.
User presses '\r', but the program receives '\n'.
'\r' char

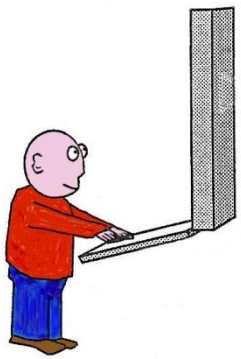

process
terminal driver
driver settings

```
unix> stty -a

speed 9600 baud; rows 24; columns 80; line = 0;
intr = ^C; quit = ^\; erase = ^?; kill = ^U; eof = ^D; eol = M-^?; eol2 = M-^?;
swtch = <undef>; start = ^Q; stop = ^S; susp = ^Z; rprnt = ^R; werase = ^W;
lnext = ^V; flush = ^O; min = 1; time = 0;
-parenb -parodd cs8 -hupcl -cstopb cread -clocal -crtscts
-ignbrk -brkint -ignpar -parmrk -inpck -istrip -inlcr -igncr icrnl ixon -ixoff
-iuclc ixany imaxbel -iutf8
opost -olcuc -ocrnl onlcr -onocr -onlret -ofill -ofdel nl0 cr0 tab0 bs0 vt0 ff0
isig icanon iexten echo echoe -echok -echonl -noflsh -xcase -tostop -echoprt
echoctl echoke
```

unix> **stty -a**

speed 9600 baud; **rows 24**; **columns 80**; line = 0;
intr = ^C; quit = ^\; **erase = ^?**; kill = ^U; **eof = ^D**; eol = M-^?; eol2 = M-^?;
swtch = <undef>; start = ^Q; stop = ^S; **susp = ^Z**; rprnt = ^R; werase = ^W;
lnext = ^V; flush = ^O; min = 1; time = 0;
-parenb -parodd cs8 -hupcl -cstopb cread -clocal -crtscts
-ignbrk -brkint -ignpar -parmrk -inpck -istrip -inlcr -igncr **icrnl** ixon -ixoff
-iuclc ixany imaxbel -iutf8
opost **-olcuc** -ocrnl **onlcr** -onocr -onlret -ofill -ofdel nl0 cr0 tab0 bs0 vt0 ff0
isig icanon iexten **echo** echoe -echok -echonl -noflsh -xcase -tostop -echoprt
echoctl echoke



```
stty columns 60
ls

stty -echo
```

NAME

tcgetattr, tcsetattr - get and set terminal attributes

SYNOPSIS

```
#include <termios.h>
#include <unistd.h>

int tcgetattr(int fd, struct termios *termios_p);

int tcsetattr(int fd, int optional_actions, const struct termios *termios_p);
```

DESCRIPTION

tcgetattr() gets the parameters associated with the object referred by fd and stores them in the termios structure referenced by termios_p.

tcsetattr() sets the parameters associated with the terminal from the termios structure referred to by termios_p. optional_actions specifies when the changes take effect:

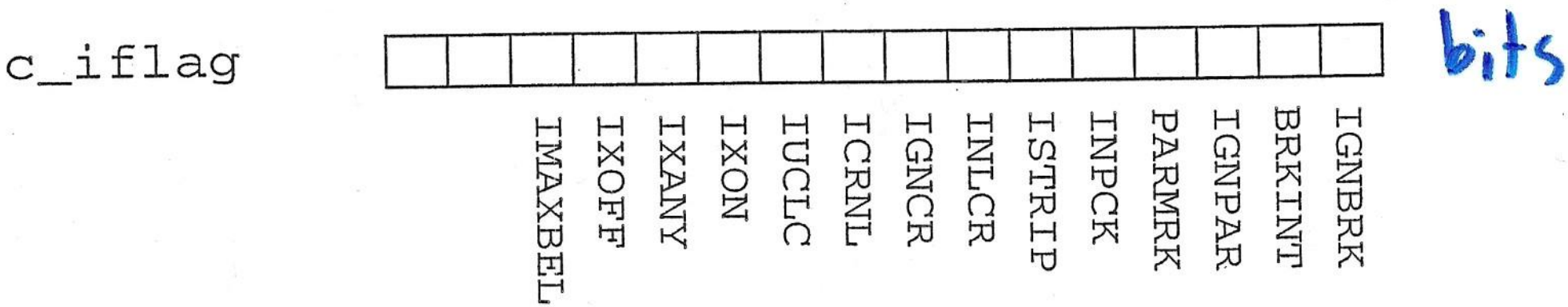
- TCSANOW the change occurs immediately
- TCSADRAIN the change occurs after all pending output is sent
- TCSAFLUSH after sending all output and discarding pending input

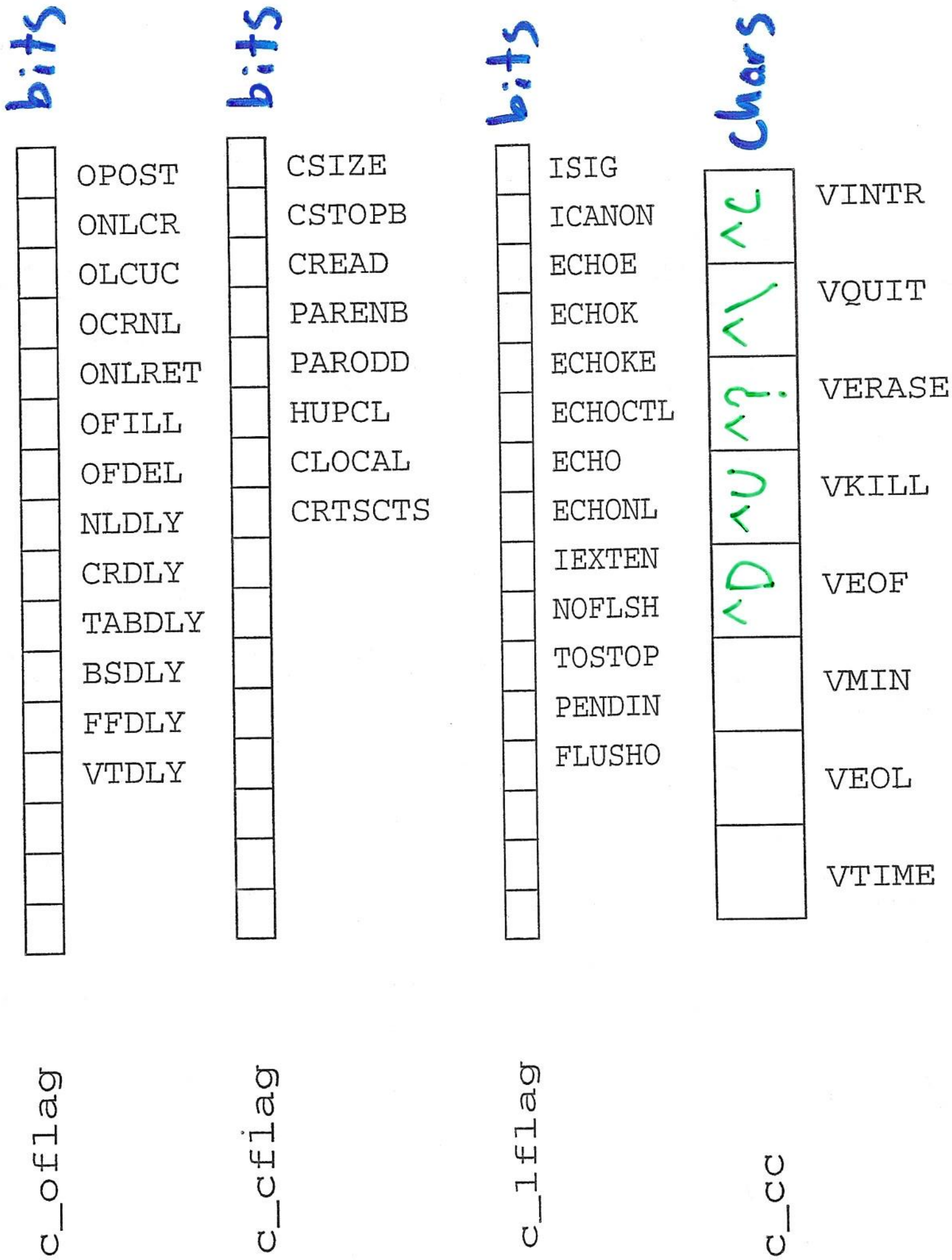
The termios functions describe a general terminal interface that is provided to control asynchronous communications ports.

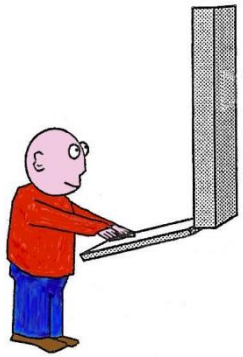
The termios structure

The termios_p argument that is a pointer to a termios structure. This structure contains the following members:

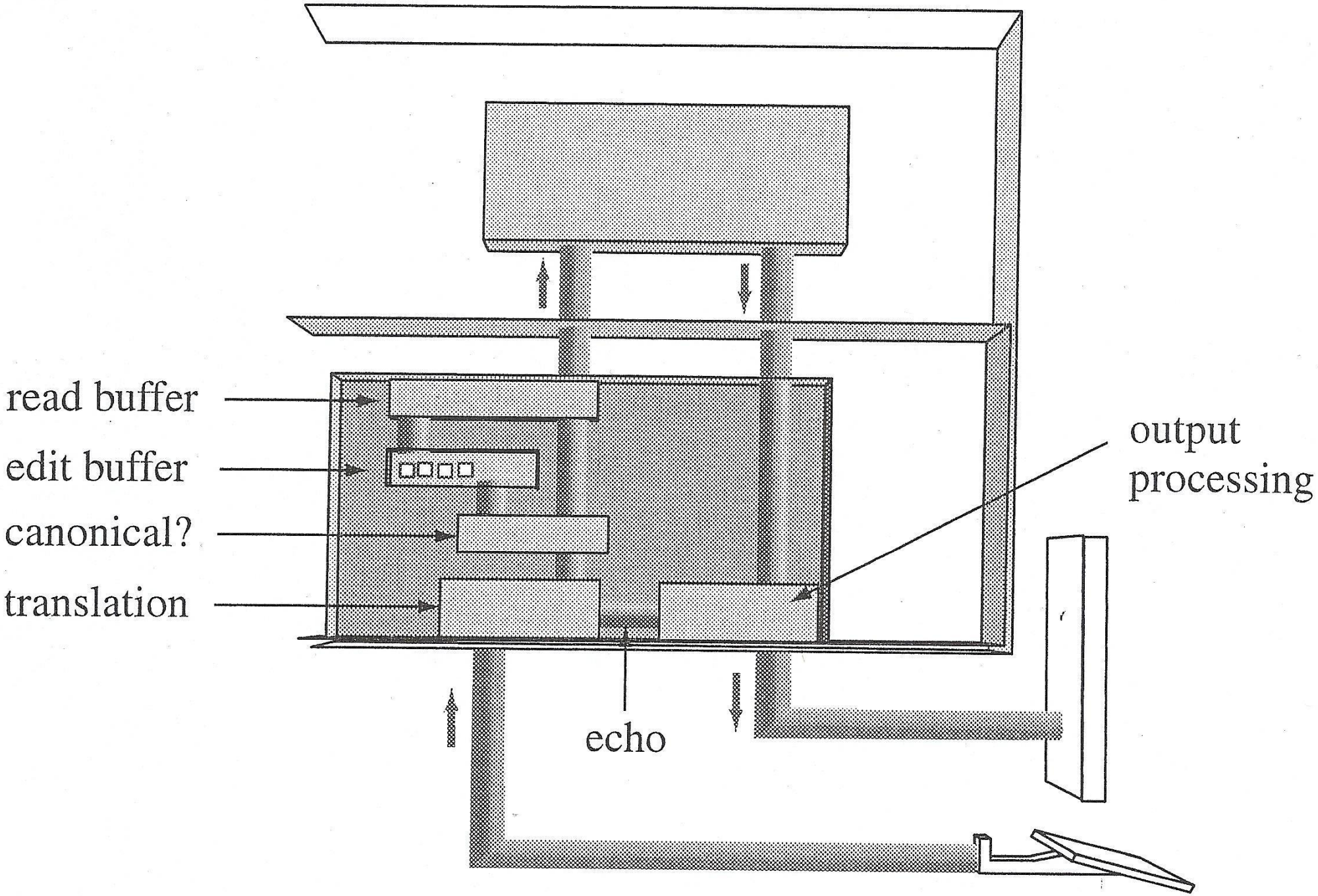
```
tcflag_t c_iflag;        /* input modes */
tcflag_t c_oflag;        /* output modes */
tcflag_t c_cflag;        /* control modes */
tcflag_t c_lflag;        /* local modes */
cc_t     c_cc[NCCS];     /* control chars */
```

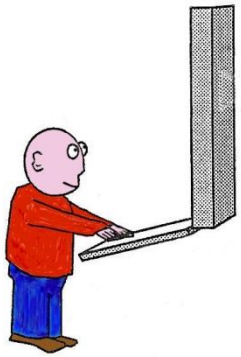



bits
OPOST
ONLCR
OLCUC
OCRNL
ONLRET
OFILL
OFDEL
NLDLY
CRDLY
TABDLY
BSDLY
FFDLY
VTDLY
c_oflag
bits
CSIZE
CSTOPB
CREAD
PARENB
PARODD
HUPCL
CLOCAL
CRTSCTS
c_cflag
bits
ISIG
ICANON
ECHOE
ECHOK
ECHOKE
ECHOCTL
ECHO
ECHONL
IEXTEN
NOFLSH
TOSTOP
PENDIN
FLUSHO
c_lflag
chars
^C
VINTR
^\
VQUIT
^?
VERASE
^U
VKILL
^D
VEOF
VMIN
VEOL
VTIME
c_cc



password1.c – turning off echo bit when entering password


read buffer
edit buffer
canonical?
translation
output
processing
echo



more

more02.c – keyboard from /dev/tty,
but problems because using canonical mode
more03.c – turning off echoing and input buffering
but problems because not erasing prompt
more04.c – erasing prompt

NAME

fcntl - manipulate file descriptor

SYNOPSIS

```
#include <unistd.h>
#include <fcntl.h>

int fcntl(int fd, int cmd, ... /* arg */ );
```

DESCRIPTION

fcntl() performs one of the operations described below on the open file descriptor fd. The operation is determined by cmd.

fcntl() can take an optional third argument. Whether or not this argument is required is determined by cmd.

. . .

commands:

F_DUPFD	make a duplicate file descriptor
F_GETFL	read file status flags
F_SETFL	set file status flags
F_SETLK	acquire or release a lock
F_GETLK	get lock status

file status flags (bits):

```
#define O_RDONLY        00
#define O_WRONLY        01
#define O_RDWR          02
#define O_APPEND        02000       // append mode
#define O_NONBLOCK      04000       // non-blocking I/O
#define O_SYNC          010000      // synchronized file update
#define O_ASYNC         020000
```

usage:

```
int settings = fcntl(fd, F_GETFL);      // get flags
settings |= O_APPEND;                   // set append mode
fcntl(fd, F_SETFL, settings);           // set flags
```

Append mode is useful if more than 1 program are trying to write to the end of a log file.

Without append mode:

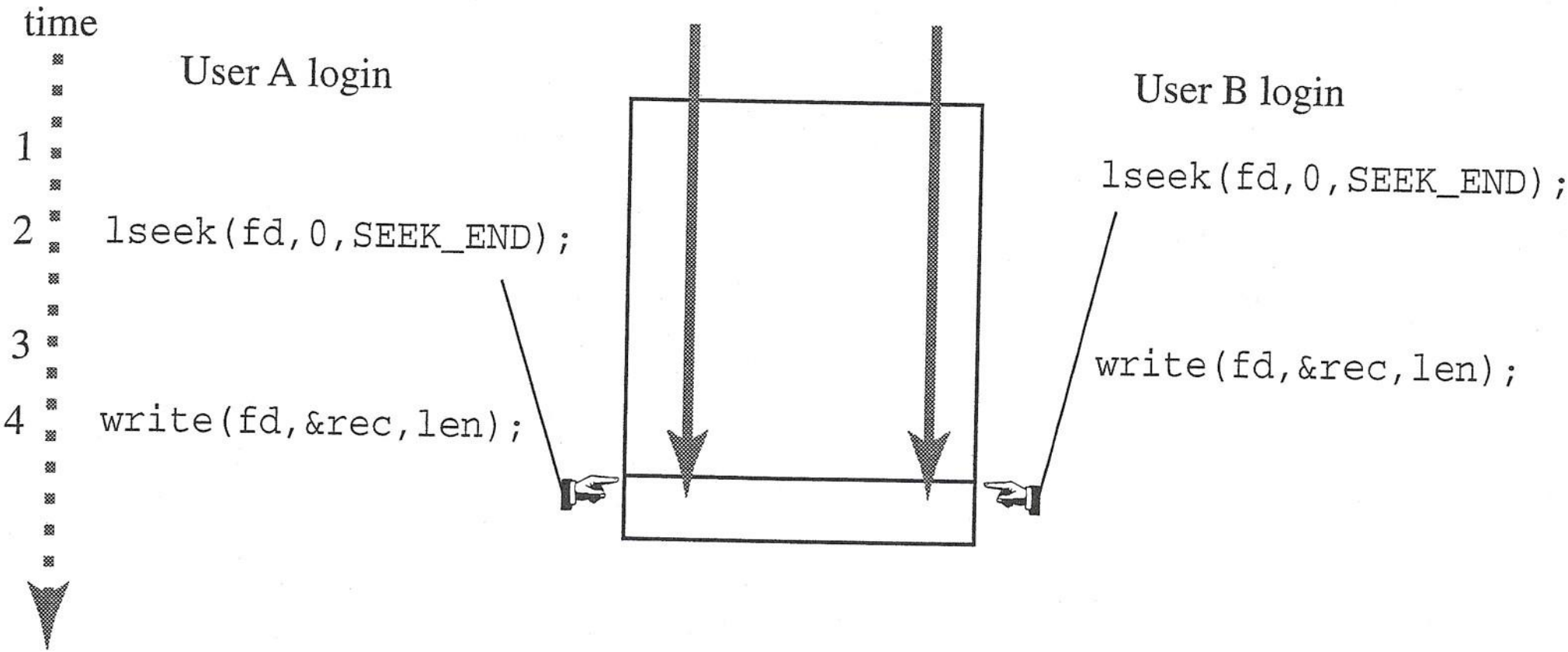


FIGURE 5.6

Interleaved `lseek` and `write` = chaos.

NAME

ioctl - control device

SYNOPSIS

```
#include <sys/ioctl.h>

int ioctl(int d, int request, ...);
```

DESCRIPTION

The ioctl() function manipulates the underlying device parameters of special files. In particular, many operating characteristics of character special files (e.g., terminals) may be controlled with ioctl() requests. The argument d must be an open file descriptor.

The second argument is a device-dependent request code.

The third argument is an untyped pointer to memory.

example:

screensize.c

```
#include <stdio.h>
#include <sys/ioctl.h>

int main() {
   struct winsize wbuf;

   if (ioctl(1, TIOCGWINSZ, &wbuf) != -1) {
      printf("chars: %d x %d\n", wbuf.ws_row, wbuf.ws_col);
      printf("pixels: %d x %d\n", wbuf.ws_xpixel, wbuf.ws_ypixel);
   }
}
```

```
unix> screensize
chars: 24 x 80
pixels: 880 x 528
```